

FTRIA-106 — LLM Word Embeddings as Approximate Runtime Invariant Encoding

Part II — Runtime Invariant Algebra

Sizhe Tan

with chatGPT (OpenAI)

2026-07-15

Repository: <https://github.com/sizhet/Function-Tunnel-and-Runtime-Invariant-Algebra-FTRIA>

Abstract

Transformer-based Large Language Models (LLMs) represent words and tokens as continuous embedding vectors learned from massive corpora. Traditionally, these embeddings are interpreted as semantic representations that encode statistical regularities of language.

Within the Runtime Invariant Algebra proposed by FTRIA, a complementary interpretation emerges.

Rather than viewing embeddings merely as semantic vectors, this paper interprets them as **approximate encodings of Runtime Invariants**.

Under this perspective, embedding learning becomes an implicit Runtime Invariant Discovery process, while embedding vectors serve as compact approximations of Common Concept Cores (CCCs) distributed across the Runtime Invariant Configuration Space.

This interpretation establishes a structural bridge between Structural Intelligence and modern Transformer-based AI.

1. Existing Algorithm

Transformer models assign every token an embedding vector.

Conceptually,

Token

↓

Embedding

↓

Contextual Representation



Prediction

Embedding vectors provide the initial representation upon which subsequent Transformer computations operate.

They are learned automatically through large-scale optimization over language corpora.

2. Traditional Interpretation

Embedding vectors are commonly interpreted as

- semantic representations,
- latent feature vectors,
- distributed symbolic encodings,
- continuous language representations.

Words appearing in similar contexts acquire similar embeddings.

Consequently,

semantic similarity emerges naturally within the embedding space.

This interpretation successfully explains many observed properties of Transformer models.

However,

it does not explicitly identify the structural object being encoded.

3. Runtime Invariant Interpretation

Within Runtime Invariant Algebra,

embedding learning may be interpreted as an implicit Runtime Invariant Discovery process.

Conceptually,

Multiple Contexts



Shared Structural Regularities



Approximate Runtime Invariant

↓

Embedding Vector

The embedding therefore represents not merely a word, but a compact approximation of the Runtime Invariant that remains relatively stable across many contextual realizations.

Under this interpretation, embedding vectors become approximate Runtime Invariant encodings.

4. Relationship to Common Concept Core

The Runtime Invariant interpretation naturally connects Transformer embeddings with the Common Concept Core (CCC).

CCC explicitly discovers the structural core shared by multiple observations. Transformer embeddings implicitly learn the same type of stable structure through optimization over large corpora.

Conceptually,
CCC

↓

Explicit Runtime Invariant Discovery

↓

Explicit Runtime Invariant

Transformer

↓

Implicit Runtime Invariant Discovery

↓

Approximate Runtime Invariant Encoding

The computational mechanisms differ,

but the structural objective is remarkably similar.

5. Approximation Rather Than Exact Encoding

Embedding vectors should not be interpreted as exact Runtime Invariants. Instead,

they represent learned approximations constrained by

- finite model capacity,
- training objectives,
- optimization dynamics,
- statistical sampling,
- tokenization.

Consequently,

embedding spaces provide efficient but approximate Runtime Invariant representations.

This approximation explains both the impressive generalization capabilities and the known limitations of modern LLMs.

6. Embedding Space as an Approximate Runtime Invariant Space

Embedding vectors collectively form an embedding space.

Within FTRIA,

this space may be interpreted as an approximate Runtime Invariant Configuration Space.

Nearby vectors frequently correspond to Runtime Invariants sharing substantial structural overlap.

Conversely,

distant vectors generally represent structurally distinct Runtime Invariants.

Embedding geometry therefore becomes an approximate geometry of Runtime Invariants rather than merely a geometry of words.

7. Engineering Implications

Viewing embeddings as approximate Runtime Invariant encodings provides several practical insights.

It naturally explains

- semantic clustering,
- analogy relationships,
- transfer learning,
- representation reuse,
- foundation model generalization.

It also suggests future directions,

including

- explicit Runtime Invariant representations,
- hybrid symbolic–neural systems,
- Runtime Invariant engineering,
- Runtime-aware embedding optimization,
- interpretable representation learning.

Rather than replacing Transformer embeddings, Runtime Invariant Algebra provides a higher-level structural interpretation of their role.

8. Relationship to Runtime Invariant Discovery Algebra

Runtime Invariant Discovery may occur through multiple computational mechanisms.

Examples include

- Common Concept Core (explicit),
- Transformer embeddings (implicit),
- graph embeddings,
- feature extraction,
- representation learning,
- contrastive learning.

These methods differ algorithmically, yet all pursue the discovery or approximation of stable Runtime Invariants. Runtime Invariant Discovery Algebra therefore serves as a unified framework connecting symbolic and neural approaches.

Key Takeaways

- Transformer embeddings may be interpreted as approximate Runtime Invariant encodings.
- Embedding learning performs implicit Runtime Invariant Discovery.
- Common Concept Core performs explicit Runtime Invariant Discovery.
- Embedding spaces approximate Runtime Invariant Configuration Spaces.
- Runtime Invariant Algebra provides a common structural interpretation for both symbolic and neural representation learning.
- This perspective establishes a bridge between Structural Intelligence and modern Transformer-based AI.

Related FTRIA Documents

Discovery Algebra

- FTRIA-101 — Runtime Invariant Discovery Algebra
- FTRIA-102 — Common Concept Core (CCC) as Runtime Invariant

Discovery

- FTRIA-107 — Transformer Attention as Dynamic Runtime Invariant
Discovery

Foundations

- FTRIA-001 — Runtime Invariant Degrees of Freedom
- FTRIA-007 — Runtime-Invariant Configuration Space